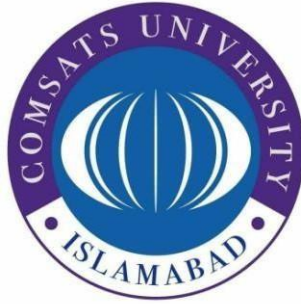


COMSATS University Islamabad

Attock Campus



Department Of Computer Science

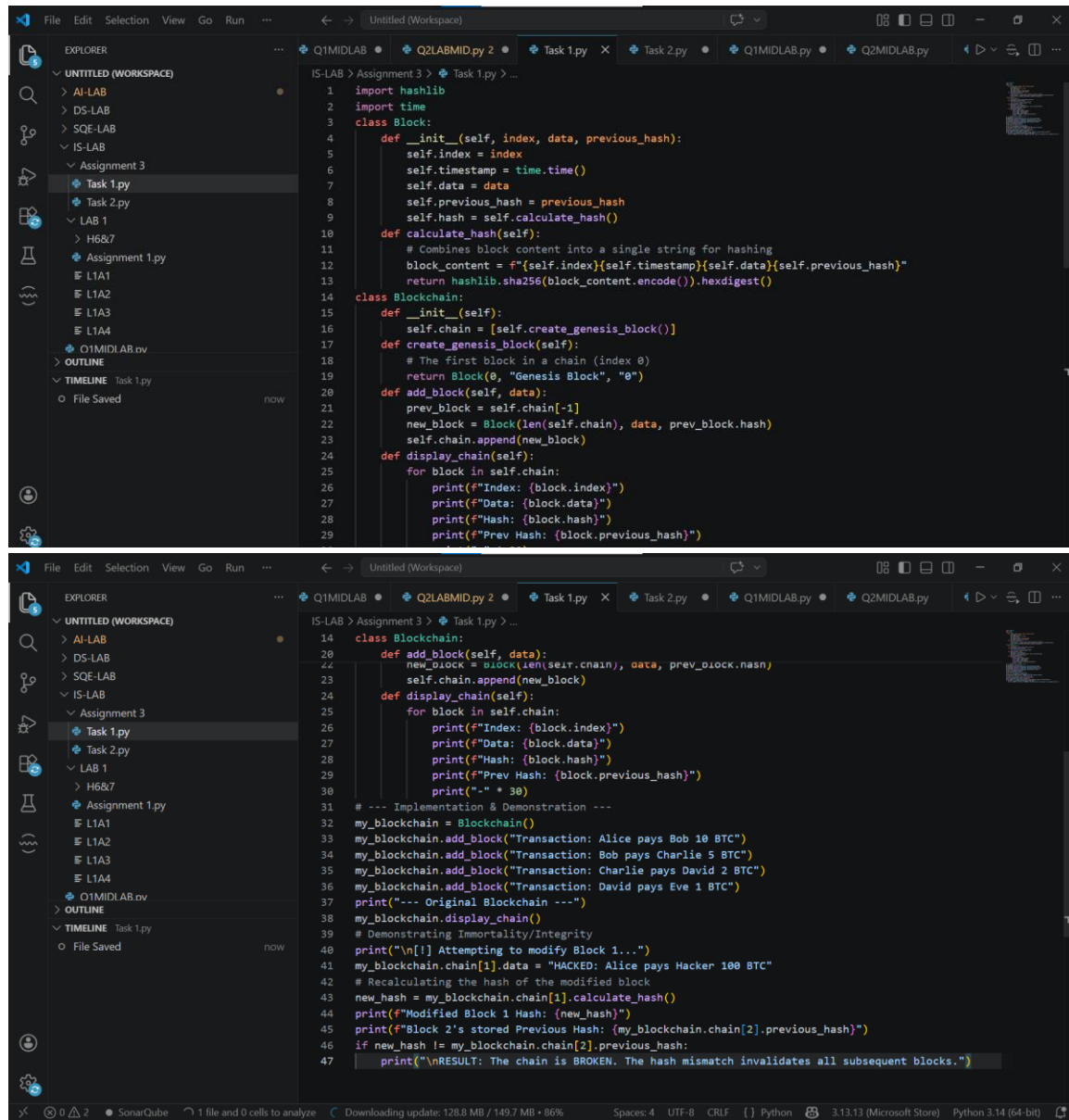
<i>Course</i>	<i>IS-LAB</i>
<i>Instructor</i>	<i>Ms. AMBREEN</i>
<i>ASSIGNMENT NO.</i>	<i>03</i>

Student Details

<i>Registration No.</i>	<i>Name</i>
<i>FA24-BSE-024</i>	<i>ISMAIL AHMED</i>

Task 1: Basic Blockchain Implementation

CODE:



```
1 import hashlib
2 import time
3 class Block:
4     def __init__(self, index, data, previous_hash):
5         self.index = index
6         self.timestamp = time.time()
7         self.data = data
8         self.previous_hash = previous_hash
9         self.hash = self.calculate_hash()
10    def calculate_hash(self):
11        # Combines block content into a single string for hashing
12        block_content = f"{self.index}{self.timestamp}{self.data}{self.previous_hash}"
13        return hashlib.sha256(block_content.encode()).hexdigest()
14 class Blockchain:
15     def __init__(self):
16         self.chain = [self.create_genesis_block()]
17     def create_genesis_block(self):
18        # The first block in a chain (index 0)
19        return Block(0, "Genesis Block", "0")
20    def add_block(self, data):
21        prev_block = self.chain[-1]
22        new_block = Block(len(self.chain), data, prev_block.hash)
23        self.chain.append(new_block)
24    def display_chain(self):
25        for block in self.chain:
26            print(f"Index: {block.index}")
27            print(f"Data: {block.data}")
28            print(f"Hash: {block.hash}")
29            print(f"Prev Hash: {block.previous_hash}")
30
31 # --- Implementation & Demonstration ---
32 my_blockchain = Blockchain()
33 my_blockchain.add_block("Transaction: Alice pays Bob 10 BTC")
34 my_blockchain.add_block("Transaction: Bob pays Charlie 5 BTC")
35 my_blockchain.add_block("Transaction: Charlie pays David 2 BTC")
36 my_blockchain.add_block("Transaction: David pays Eve 1 BTC")
37 print("--- Original Blockchain ---")
38 my_blockchain.display_chain()
39 # Demonstrating Immutability/Integrity
40 print("\n[] Attempting to modify Block 1...")
41 my_blockchain.chain[1].data = "HACKED: Alice pays Hacker 100 BTC"
42 # Recalculating the hash of the modified block
43 new_hash = my_blockchain.chain[1].calculate_hash()
44 print(f"Modified Block 1 Hash: {new_hash}")
45 print(f"Block 2's stored Previous Hash: {my_blockchain.chain[2].previous_hash}")
46 if new_hash != my_blockchain.chain[2].previous_hash:
47     print("\nRESULT: The chain is BROKEN. The hash mismatch invalidates all subsequent blocks.")
```

OUTPUT:

```
PS D:\IS-LAB> & "C:/Users/ISMAIL AHMED/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "d:/IS-LAB/Assignment 3/Task 1.py"
--- Original Blockchain ---
Index: 0
Data: Genesis Block
Hash: f8e5ac156c2e8fb8a5784b2dad2d396866a19b3461a4d4cfaab91594fe18c53e
Prev Hash: 0
-----
Index: 1
Data: Transaction: Alice pays Bob 10 BTC
Hash: c5092ae8109e46d9b80fe453d5d4ae90bdd495231eb668e412219c2b6d2a4500
Prev Hash: f8e5ac156c2e8fb8a5784b2dad2d396866a19b3461a4d4cfaab91594fe18c53e
-----
Index: 2
Data: Transaction: Bob pays Charlie 5 BTC
Hash: 734ba8390841397271c8660eba2f5b0a98f528ff21085b246598c63e71a52b97
Prev Hash: c5092ae8109e46d9b80fe453d5d4ae90bdd495231eb668e412219c2b6d2a4500
-----
Index: 3
Data: Transaction: Charlie pays David 2 BTC
Prev Hash: 734ba8390841397271c8660eba2f5b0a98f528ff21085b246598c63e71a52b97
-----
Index: 4
Data: Transaction: David pays Eve 1 BTC
Hash: 37578ae1693a9469de855d788a6f81170fc505c0ac726eacab21ed779dafc598
Prev Hash: c5fed850fbec12a9afb7cca5d418848097709ed67504857740abdbb3e12dd6a6
-----
[!] Attempting to modify Block 1...
Modified Block 1 Hash: b44e7b32157c3a59ce0b4c55dbaf068182a8aaccd122c11ffe1d6a8f92f0f2ec
Block 2's stored Previous Hash: c5092ae8109e46d9b80fe453d5d4ae90bdd495231eb668e412219c2b6d2a4500
RESULT: The chain is BROKEN. The hash mismatch invalidates all subsequent blocks.
```

Task 2: Elliptic Curve Cryptography (ECC) Basics

CODE:

```
1 # Parameters for a small curve: y^2 = x^3 + 2x + 2 (mod 17)
2 P = 17
3 A = 2
4 B = 2
5 def point_addition(P1, P2, a, p):
6     if P1 is None: return P2
7     if P2 is None: return P1
8     x1, y1 = P1
9     x2, y2 = P2
10    if x1 == x2 and y1 != y2:
11        return None # Point at infinity
12    if P1 == P2:
13        # Point Doubling
14        m = (3 * x1**2 + a) % pow(2 * y1, -1, p)
15    else:
16        # Standard Addition
17        m = (y2 - y1) * pow(x2 - x1, -1, p)
18    x3 = (m**2 - x1 - x2) % p
19    y3 = (m * (x1 - x3) - y1) % p
20    return (x3, y3)
21 def multiply(P1, k, a, p):
22     # Scalar multiplication: k * P
23     result = None
24     addend = P1
25     for bit in bin(k)[2:][::-1]:
26         if bit == '1':
27             result = point_addition(result, addend, a, p)
28             addend = point_addition(addend, addend, a, p)
29     return result
30 # --- Key Generation ---
```

```

30 # --- Key Generation ---
31 # Generator Point G
32 G = (5, 1)
33 private_key = 3 # Chosen randomly
34 public_key = multiply(G, private_key, A, P)
35 print(f"ECC Key Pair Generated:\nPrivate Key: {private_key}\nPublic Key: {public_key}")
36 # --- Simple EC-ElGamal Encryption ---
37 msg_point = (9, 1) # A message mapped to a point on the curve
38 k_random = 2 # Random value for encryption
39 # Ciphertext (C1, C2)
40 c1 = multiply(G, k_random, A, P)
41 shared_secret = multiply(public_key, k_random, A, P)
42 c2 = point_addition(msg_point, shared_secret, A, P)
43 print(f"\nEncrypted Message Point: ({c1}, {c2})")
44 # --- Decryption ---
45 # Decryption: msg = C2 - (private_key * C1)
46 s = multiply(c1, private_key, A, P)
47 # Subtracting is adding the point with negative y
48 s_neg = (s[0], -s[1] % P)
49 decrypted_msg = point_addition(c2, s_neg, A, P)
50 print(f"Decrypted Message Point: {decrypted_msg}")

```

OUTPUT:

The screenshot shows a Python IDE with a terminal window. The terminal output matches the code execution:

PS D:\IS-LAB> "C:/Users/ISMAIL AHMED/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "d:/IS-LAB/Assignment 3/Task 2.py"

ECC Key Pair Generated:

Private Key: 3

Public Key: (10, 6)

Encrypted Message Point: ((6, 3), (5, 1))

Decrypted Message Point: (9, 1)

PS D:\IS-LAB>

Security Analysis:

1. **Blockchain Integrity (Task 1):** The security relies on the Avalanche Effect. Even a single bit change in a block's data results in a completely different hash. Since each block stores the hash of the preceding block, a change in block invalidates the previous_hash of block , effectively breaking the chain.
2. **ECC Robustness (Task 2):** ECC security is based on the Elliptic Curve Discrete Logarithm Problem (ECDLP). While it is easy to perform scalar multiplication (), it is computationally infeasible to find if only and are known. This allows for smaller key sizes compared to RSA while maintaining equivalent security levels.